

RS004 Week 3 — English Worksheet

Topic: Player & Keyboard Movement · **Course:** Platformer Game · **Time:** about 45 minutes

This week a **player** appears — a rectangle you control with the **arrow keys**. In the live lesson you wrote the code for it. On this worksheet you will think about and explain that code: how `pygame.Rect` tracks position, how `pygame.key.get_pressed()` reads held-down keys, and how the player is stopped from sliding off the screen.

Keep these words handy: **Rect**, **position**, `get_pressed()`, **speed**, **boundary (edge)**.

1 · Predict 🧠

Read each snippet. Write what you think it does.

```
player = pygame.Rect(100, 500, 50, 50)
```

What do the four numbers mean? Where on the screen is the player?


```
keys = pygame.key.get_pressed()
if keys[pygame.K_RIGHT]:
    player.x = player.x + 5
```

The right arrow is held down. What happens to the player each frame?


```
if player.x > WIDTH - player.width:
    player.x = WIDTH - player.width
```

The player runs into the right wall. What does this code stop from happening?

2 · Rect Reading

A player is `pygame.Rect(100, 500, 50, 50)` . Fill in the blanks.

```
player.x      = ____  
player.y      = ____  
player.width  = ____  
player.right  = ____ (hint: x + width)  
player.bottom = ____ (hint: y + height)
```


3 · Spot the Bug 🐛

Each snippet is broken. Write the fixed code, then explain why the original was wrong.

Bug A — Arrow keys should move the player smoothly while held. Right now the player only moves on a brand-new key press, one step at a time.

```
for event in pygame.event.get():
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RIGHT:
            player.x = player.x + 5
```

Hint: for *held* keys we use `pygame.key.get_pressed()`, not `KEYDOWN` events.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — The player should stop at the left edge. Right now it disappears off the left side.

```
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]:
    player.x = player.x - 5
```

Hint: after moving, check if `player.x` went below 0 and pull it back.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — Both arrows should work. Right now only the right arrow does anything.

```
keys = pygame.key.get_pressed()
if keys[pygame.K_RIGHT]:
    player.x = player.x + 5
    if keys[pygame.K_LEFT]:
        player.x = player.x - 5
```

Hint: the left check is trapped *inside* the right check. They should be separate.

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Explain the Code

Here is a working player. Read it carefully and answer the questions below.

```
PLAYER_SPEED = 5
player = pygame.Rect(100, 500, 50, 50)

keys = pygame.key.get_pressed()

if keys[pygame.K_RIGHT]:
    player.x = player.x + PLAYER_SPEED
if keys[pygame.K_LEFT]:
    player.x = player.x - PLAYER_SPEED

if player.x < 0:
    player.x = 0
if player.x > WIDTH - player.width:
    player.x = WIDTH - player.width

pygame.draw.rect(screen, (255, 0, 0), player)
```

What does `PLAYER_SPEED` control, and what would happen if you made it bigger?

Why are the two arrow-key checks both `if` (not `if / elif`)? What does that let the player do?

The line `if player.x < 0: player.x = 0` — in your own words, what is it protecting against?

What does `WIDTH - player.width` mean, and why not just use `WIDTH` ?

What is the last line doing, and what would the player look like without it?

5 · Explain Your Lesson Code 🎥

Explain the code **you** wrote in today's lesson. Record a short phone video — you may show it running while you talk. Try to use these words: **Rect, arrow keys, speed, edge, boundary.**

1. Show your player moving left and right.
2. Run into a wall and show it stops.
3. Read your boundary check out loud and explain what it does.
4. Point to where you set the speed and say what number you chose and why.

Write what you will say in your video. Plan it here before you record.

Submit 

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.